EE1003 Introduction to Computer I

Programming Assignment 2: Positive Binary Adder

Due Date: 2021/11/14(Sun.) 23:59

Background

With the rapid development of technologies today, computers have become indispensable device of daily life of general publics. Many complex problems such as statistics calculations can be solved by computers. As you know, the computer is constructed by binary system where only 0s and 1s can be processed by the computer hardware. Therefore, all information should be encoded into binary format before the computer can process them. Even the basic decimal addition should be translated to binary additions. In this problem, you are asked to write a **recursion program** to simulate the way computer performs addition calculation.

Description

In this program, you need to simulate the behavior of a binary adder. The binary adder will firstly take two decimal integers as inputs, translate the decimal integers into binary ones, perform and show the addition process in binary, and show the partial sum in decimal. Then the adder will take another decimal integer as an input, translate the decimal integers into binary one, add this number to the partial sum in binary, and show the partial sum in decimal. The process repeats until a sentinel value is encountered. Note that you MUST to use recursion on “decimal to binary translation” function, “binary adder” function, and “binary to decimal translation” function, otherwise you will get 0 points. All computations should be computed with binary operation. Directly using decimal operations “+” or “-” is not allowed in this PA (bonus included).

The program will operate as follows:

1. When the program starts, a welcome message is shown and prompt the user to enter the first decimal number (a positive integer) as an augend.
2. Then, the program will prompt the user to enter the decimal number (a positive integer) as an addend.
3. The program will translate the two numbers to its binary form, and perform the addition. The program will show the process by a single-line equation.
4. After that, the program will display the partial sum in decimal form.

5. Then, the program prints a blank line, then asks the user to enter a decimal number (a positive integer) again as an addend.
6. The program will translate the addend entered in step 5 to its binary form, and perform the addition. The program will show the process in a single-line equation.
7. After that, the program will display the partial sum in decimal form.
8. The program will repeat the procedure of step 5 to step 7 until a sentinel value is encountered.
9. If the program encounters a sentinel value, -1, during any input step, the system will show the word "Bye" and end its execution.
10. Your program needs to provide fool-proof mechanisms to avoid program errors. If the user enters something which are not defined, the program needs to ask the user to re-enter the correct information.
11. In this program, all global variables are not allowed. For local variables, only data type **int**, **long int**, and **long long int** are allowed. **Static** variables are not allowed. Please store the decimal number entered by a user in a variable of data type **int**, and the converted binary number in a variable of data type **long long int**. In case the binary number converted from the user-provided decimal number is out of range (i.e., the bunary number is not able to be stored in a variable), the program will show an error message and ask the user to enter a decimal number again.
12. Note that you have to write recursion functions for decimal to binary

Input

Positive integers of range [0~2^{31}-1] in decimal format, or -1 to terminate the program.

Output

The program will show a welcome message and ask the user to enter a decimal number. Then, the program will ask the user to enter another decimal number. Then the program will output the calculation process of the binary addition in a single-line equation. After that, the program will display the partial sum in decimal. When the input decimal number or the partial sum is out of range, the program will show an error message and ask the user to enter again. When the program obtains -1 as the input, the program will show "Bye" and end its execution. Note that all the output information about the user-provided decimal number should be shown with its ordinal information (i.e., 1st, 2nd, …).

```
*** Welcome to Positive Binary adder ***
Please enter the 1st number in decimal: -20
The decimal number you entered is out of range. Please enter the 1st
decimal number again: -255
The decimal number you entered is out of range. Please enter the 1st
decimal number again: 8
Please enter the 2nd number in decimal: -77
The decimal number you entered is out of range. Please enter the 2nd
decimal number again: -88
The decimal number you entered is out of range. Please enter the 2nd
decimal number again: 19
Calculating... 1000 + 10011 = 11011
Partial Sum: 27

Please enter the 3rd number in decimal: 2147483647
The decimal number you entered is out of range. Please enter the 3rd
decimal number again: 152341
Calculating... 11011 + 100101001100010101 = 100101001100110000
Partial Sum: 152368

Please enter the 4th number in decimal: 412386
The partial sum is out of range. Please enter the 4th decimal number
again: 999
Calculating... 100101001100110000 + 1111100111 = 100101011100010111
Partial Sum: 153367

Please enter the 5th binary number: -1
Bye
```

Bonus

Add the binary subtraction function by taking the negative input decimal numbers. In the bonus, we will apply two's complement to represent negative numbers. To simplify the difficulty, please express all the negative binary numbers with the least bits that can represent the negative binary number. For example, -5 will be represented as 1011 instead of 11111011. Moreover, the positive binary number will be represented by adding a sign bit 0 in the front. For example, 8 will be represented as 01000 instead of 1011.

Here we assume 0 will NOT be used as an input decimal number and will use it as the sentinel value. The range of the input decimal numbers is $[-2^{31}\sim2^{31}-1]$.

We provide a sample of bonus version: (The red words are the parts that need to be changed.)

[Hint] The operation of 2's completement need to perform sign extension. However, you don't need to show the number with sign extensions on the output.

```
*** Welcome to Binary Adder and Subtractor ***
Please enter the 1st number in decimal: 8
Please enter the 2nd number in decimal: 19
Calculating... 01000 + 010011 = 011011
Partial Sum: 27

Please enter the 3rd number in decimal: 2147483647
The decimal number you entered is out of range. Please enter the
3rd decimal number again: 50
Calculating... 011011 + 0110010 = 01001101
Partial Sum: 77

Please enter the 4th number in decimal: -222222
Calculating... 01010001 + 1001001101111110010 = 10010011100011111 1
Partial Sum: -222145

Please enter the 5th number in decimal: -444444
The decimal number you entered is out of range. Please enter the
4th decimal number again: -1
Calculating... 10010011100011111 1 + 11 = 100100111000011110
Partial Sum: -222146

Please enter the 6th number in decimal: 0
Bye
```

Submission Requirement

You have to submit a source code file (not the entire project) named as StudID_PA2.cpp (ex: 9862534_PA2.cpp) and a report named StudID_Name_PA2_report.pdf (ex: 9862534_ 陳 聿 廣 _PA2_report.pdf). If you implement a bonus version, please separate it from the original version. Name your source code file of the bonus version as StudID_PA2_bonus.cpp (ex: 9862534_PA2_bonus.cpp) and upload to ee-class with the original version. Note that the only acceptable report file format is .pdf, no .doc/.docx or other files are acceptable. **BE SURE to follow the naming rule mentioned above. Otherwise, your program will be not graded.**

We don't restrict the report format and length. In your report, you have to at least describe:

1. How to compile and execute your program; (You can use screenshot to explain)
2. The completion of the assignment; (If you complete all requirements, just specify all)
3. The hardness of this assignment and how you overcome it;
4. Bonus function(s) you implement if any
5. Any suggestions about this programming assignment?

You can also put anything related to the PA in your report, such as pseudocode, control flow diagram, programming developing thought, etc.

Your program will be judged with Code::Blocks 20.03 and GNU GCC Compiler (MinGW-W64 project version 8.1.0, 32/64 bit, SHE). We will use C++ 11 standard.

Grading

The grading is as follows:

(1) Correctness of your code: 50%
(2) Readability of your code: 10%
(3) The report: 10%
(4) Demo session: 30%
(5) Bonus (at most): 10%

Please submit your assignment on time. Otherwise, the penalty rule will apply:

- Within 72hrs delay: 20% off
- More than 3 days: 0 point

Be sure to attend a demo session (the time will be announced later). If you have questions, please E-mail to both me (andyygchen@ee.ncu.edu.tw) and TA 林俐秀 (sophia30516@gmail.com)

Reference

[1] Chapter 2 ~ Chapter 5, Paul Deitel and Harvey Deitel, "C++ how to program late objects version," 7th edition, Person 2011.

[2] How to convert decimal number to binary number: https://youtu.be/C5Gra1GhsoI

[3] Tow's complement: https://youtu.be/631hXbD1iSw